

Divide and Conquer: Role Assignment and Coordination Strategies in LLM-Powered Multi-Agent Systems

Maryam Khalid

*Department of Data Science and Artificial Intelligence
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i221917@nu.edu.pk*

Laiba Zubair

*Department of Data Science and Artificial Intelligence
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i221888@nu.edu.pk*

Abstract—As large language models grow more capable, researchers are building systems where multiple LLM agents collaborate on tasks too complex for any single model. One of the core challenges is assigning agents clear roles and keeping them coordinated through synchronized actions. This paper presents the design, implementation, and evaluation of a custom multi-agent orchestration system in which three agents—Planner, Researcher, and Writer—work with fixed roles under the supervision of a Judge agent that controls task progression through a quality score. The system integrates a Wikipedia-based retrieval tool for grounded fact lookup and a fault-tolerant retry mechanism that injects Judge feedback into context on failure. Experiments across five diverse knowledge tasks (a single end-to-end run recorded in `eval_results.json`, not an average over random seeds) yield a mean judge score of 8.87/10 across all agent stages (15 scores), with a 100% pipeline completion rate and self-correcting behavior demonstrated through retry with context augmentation. Results are tied to that run and to Groq API behavior (rate limits, response times). We analyze trade-offs of deterministic role assignment, document tool and API failure modes, and identify improved Wikipedia title resolution and adaptive role prompting as directions for future work.

Index Terms—multi-agent systems, large language models, role assignment, coordination, autonomous agents, fault tolerance, tool-use, orchestration

I. INTRODUCTION

In recent years, large language models (LLMs) have progressed beyond traditional question-answering systems. Models such as GPT-4 and Claude can now plan and reason across long contexts, generate computer code, and reflect on their own outputs, which has created a distinct new design space. Rather than building one general model for all applications, we can establish a team of specialized agents that cooperate to complete a task [1].

That idea—multi-agent systems (MAS) powered by LLMs—has attracted significant research attention. But building these systems is harder than it sounds. Multi-agent coordination has been studied for decades in classical AI [2], yet LLM-based agents bring a different set of properties altogether. They communicate in natural language rather than structured signals, interpret and generate free-form instructions, and their

capabilities are not fixed—they depend heavily on prompting and available context.

This creates a central design problem: how do you decide which agent does what, and how do you keep them coordinated? If roles are too rigid, the system cannot adapt when something goes wrong. If roles are too loose, agents start duplicating work or contradicting each other [3]. In practice, both failure modes are common, and neither is trivial to fix.

Frameworks like AutoGen [4], MetaGPT [5], and CAMEL [6] have each shown real promise on specific benchmarks. Yet none of these systems fully addresses the triple challenge of flexibility, coherence, and fault tolerance simultaneously—a gap that motivates the system designed in this work.

This paper makes the following contributions. First, we design and implement a complete multi-agent orchestration system from scratch, without relying on existing frameworks such as LangChain or CrewAI. Agents are assigned fixed roles—Planner, Researcher, and Writer—in a deterministic pipeline, gated by a Judge agent that scores each output on a scale of 1–10. Second, we integrate an external Wikipedia retrieval tool that the Researcher agent can invoke autonomously via a structured tag interface. Third, we evaluate the system across five diverse tasks and record per-stage judge scores, retry counts, tool invocation rates, and task completion times, cross-checked against machine-readable logs.

II. RELATED WORK

A. Background: Classical Multi-Agent Systems

Multi-agent coordination has been studied as part of AI research for many decades. Traditional approaches such as contract-net protocols and auction-based mechanisms allowed agents to bid for tasks based on local capabilities and available resources [2]. These approaches worked well in structured, well-defined task environments—a key limitation that LLM-based systems are designed to overcome.

Multi-agent reinforcement learning (MARL) [7] took a different route, training agents to coordinate through shared reward signals and learned communication channels. MARL

TABLE I
COMPARISON OF LLM-BASED MULTI-AGENT FRAMEWORKS
(ILLUSTRATIVE)

Framework	Role Type	Fault Tol.	Tool Use
MetaGPT [5]	Static	None	Limited
AutoGen [4]	Dynamic	Partial	Yes
CAMEL [6]	Semi-fixed	None	No
This Work	Static	Yes	Yes

has produced impressive results in simulation and game environments, but generalizing to open-ended, language-driven tasks remains an unsolved challenge.

B. Role Assignment in LLM-Based Frameworks

The most influential recent frameworks each propose a different answer to the role assignment problem. MetaGPT [5] assigns agents to fixed roles inspired by software engineering teams—each governed by a role-specific system prompt. AutoGen [4] defines agents by conversational role and permits them to spawn sub-agents on demand. Chen et al. [3] documented that loosely structured systems frequently suffer from role confusion—agents duplicating sub-tasks or leaving critical work unowned. CAMEL [6] structures interaction around a fixed instructor-executor pair, reducing ambiguity without requiring predefined task pipelines.

Table I summarizes key dimensions along which these frameworks differ.

C. Coordination and Evaluation

Surveys of LLM-based autonomous agents [1] identify three recurring open problems: coordinating the interaction of multiple agents, managing shared memory between agents, and handling responsibility hand-offs cleanly. AgentBench [8] evaluates LLMs as agents across diverse environments; our work is complementary in that it instruments an explicit per-stage judge, scored retries, and transparent logging in a Python implementation suited to course-scale replication.

III. PROBLEM IDENTIFICATION AND MOTIVATION

Complex knowledge synthesis tasks—such as writing a research report or summarizing a large multi-dimensional topic—benefit from decomposition into specialized sub-tasks. A single agent asked to plan, research, and write simultaneously tends to produce outputs that are either shallow in detail or poorly structured, because it cannot focus exclusively on any one responsibility at a time.

The core problem this work addresses is: *how can an LLM-based system decompose a complex knowledge task, execute each sub-task reliably, verify quality autonomously, and recover from failures without human intervention?*

Solving this problem requires multi-step reasoning and planning, role-specific execution with dedicated context per agent, quality judgment under uncertainty, autonomous error recovery through feedback-augmented retry, and optional external tool use for grounded information retrieval. A single-agent prompt cannot satisfy all of these simultaneously; a

System architecture

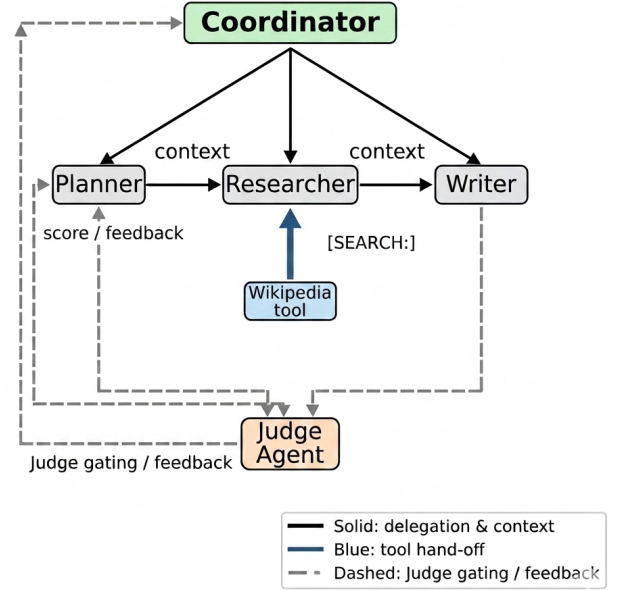


Fig. 1. System architecture. Solid arrows: delegation and context. Dashed arrows: Judge feedback and gating. Blue link: Researcher–Wikipedia [SEARCH:] hand-off.

human-in-the-loop workflow can, but defeats the purpose of autonomous operation at scale. The architecture described in this paper targets these requirements directly.

IV. METHODOLOGY AND SYSTEM ARCHITECTURE

A. Overall Architecture

The system consists of four components: (1) a Coordinator (orchestrator); (2) three task agents—Planner, Researcher, and Writer; (3) a Judge agent; and (4) an external tool interface. Figure 1 illustrates the architecture and the flow of execution.

B. Agent Roles

Each agent is an instance of a base `Agent` class with a unique role-specific system prompt and a shared LLM client. When an agent executes, it receives both the original task and the accumulated context from all preceding agents.

Planner: Produces a structured plan covering sub-tasks, research directions, and a writing strategy.

Researcher: Synthesizes information informed by the plan. It may emit [SEARCH: keyword]; the Coordinator intercepts this tag, calls the Wikipedia tool, and appends the observation to context before optionally re-running the Researcher.

Writer: Produces the final user-facing answer from the accumulated context. The Writer is instructed to omit internal tool markup from its output.

Judge Agent: Outputs JSON only, with three fields: `score` (integer 1–10), `feedback` (string), and `passed` (true if

score ≥ 7). The Coordinator parses this structure to decide whether to advance the pipeline or trigger a retry.

C. Orchestration and Fault Tolerance

The Coordinator executes agents in the fixed sequence Planner $\rightarrow$ Researcher $\rightarrow$ Writer. After each agent output, the Judge evaluates it. The orchestrator uses the Judge’s `passed` field from the parsed JSON to decide gating, not a separate recomputation of `score  $\geq 7$` in code; a malformed or inconsistent JSON response is treated as failure and may trigger retry.¹ If `passed` is false, Judge feedback is appended to context and the same agent is re-run, up to three attempts per stage. If a stage exhausts attempts, the pipeline aborts and returns partial state. Figure 2 summarizes this logic.

The retry mechanism serves two distinct purposes. First, it addresses *content quality failures*, where the Judge scores an agent output below the passing threshold of 7 because the response is incomplete, off-topic, or poorly structured. In these cases, the Judge’s textual feedback is injected directly into the shared context before the next attempt, giving the agent an explicit signal describing what to fix. This is not a simple re-run of the same prompt; the context is strictly richer on each retry, making it more likely the agent converges toward a passing output. Second, it addresses *infrastructure failures*, where transient API behavior—such as an HTTP 429 rate-limit response from Groq during a Judge call—causes the Judge to return malformed JSON that cannot be parsed. In this case the orchestrator falls back to a default score of 5 with `passed = false`, which forces a retry even though the underlying agent output may have been acceptable. This coupling between evaluation reliability and provider-side rate limits is an inherent limitation of the single-run experimental setup and is discussed further in Section VII.

The maximum of three attempts per stage was chosen as a practical bound: it allows genuine self-correction without the risk of an infinite loop, and it keeps total API call overhead bounded. In the evaluation run reported in this paper, only one retry was observed (Task T4, Researcher stage), confirming that the threshold is rarely reached under normal operating conditions. The pipeline never aborted in any of the five evaluated tasks, yielding a 100% completion rate. When a retry does occur, the Coordinator logs both the attempt count and the Judge’s feedback string, making the correction pathway transparent and auditable.

It is worth distinguishing the retry loop from the tool-augmentation step that also re-runs the Researcher. When the Researcher emits a `[SEARCH: keyword]` tag, the Coordinator intercepts it, calls the Wikipedia tool, appends the observation to context, and re-runs the Researcher *regardless of whether the previous output passed the Judge*. This tool re-run does not count against the three-attempt retry budget; it is a context-enrichment step that precedes Judge evaluation for that attempt. Only a subsequent Judge rejection would increment

¹In practice, `passed` and `score` almost always align with the system prompt, but a rare parse error on the Judge side can still force a re-attempt.

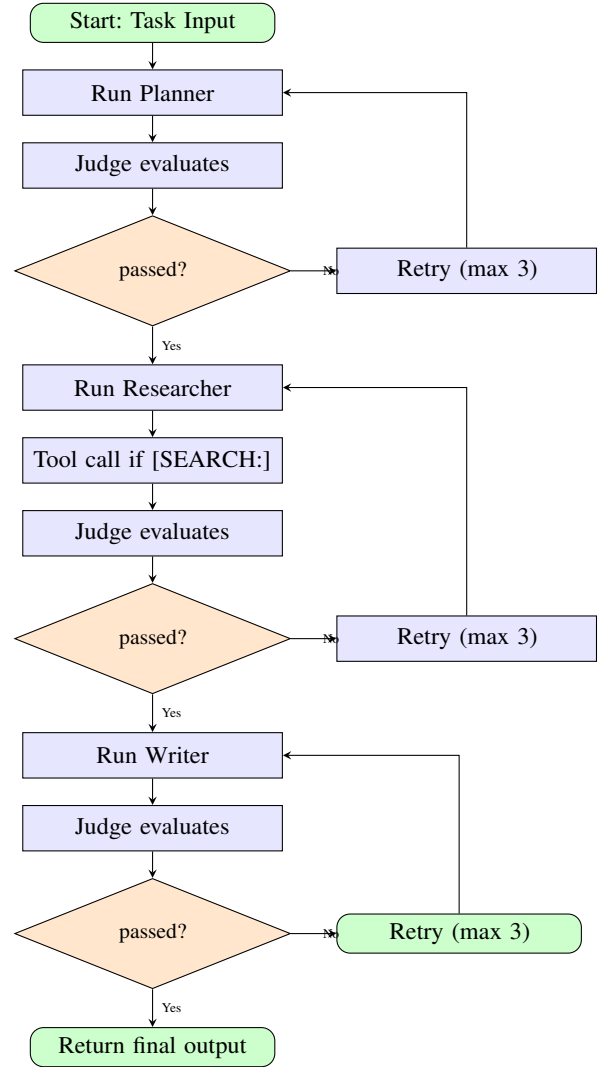


Fig. 2. Orchestration flow with fault-tolerant retry loop. Each stage allows up to three attempts before the pipeline aborts.

the retry counter. The two mechanisms are therefore orthogonal: tool augmentation enriches context proactively, while the retry loop corrects quality reactively based on Judge feedback.

D. Tool Integration

The Wikipedia helper calls the REST `/page/summary/{title}` endpoint (implemented in the project as a title-based lookup). Natural-language phrases often do not correspond to exact page titles, which yields HTTP 404 responses whose error strings are still injected into context so the Researcher can adapt.

E. LLM Backend

All agents call Groq’s hosted `llama-3.1-8b-instant` model through an OpenAI-compatible client (GroqLLM). The client uses `temperature=0.7` and `max_tokens=2048`, and sleeps two seconds before each API request to reduce rate-limit errors on the free tier.

TABLE II
EVALUATION TASKS USED IN EXPERIMENTS

Task ID	Task Description
T1	Write a report on the causes and effects of climate change
T2	Explain the pros and cons of nuclear energy
T3	Summarize the economic impact of COVID-19
T4	Describe the history and future of artificial intelligence
T5	Analyze the causes of World War I

V. DATASET, TOOLS, AND APIS

A. Evaluation Tasks

Five open-domain tasks were defined in `evaluate.py`; they match Table II.

B. APIs and Libraries

Groq API: OpenAI-compatible chat completions for `llama-3.1-8b-instant`.

Wikipedia: Summary retrieval used only from the Researcher’s `[SEARCH:]` path.

Python: `openai`, `python-dotenv`, standard library `re`, `json`, `time`.

VI. EXPERIMENTAL SETUP AND METRICS

A. Experimental Protocol

The script `evaluate.py` executes all five tasks sequentially. For each task the script records per-stage judge scores, attempt counts, tool invocation and success flags, and wall-clock time. The numbers reported here correspond directly to the run stored in `eval_results.json`. Sampling is conducted at a fixed temperature of 0.7 and is not swept across other values. Because stochastic sampling and external API interactions (e.g., transient rate-limit responses) can change scores, tool paths, and retry counts, the results should be understood as a single execution sample, not as a multi-seed average.

B. Metrics

Judge score (per stage): Integer 1–10 from the Judge for each of the Planner, Researcher, and Writer stages.

Pass rate: Fraction of stages that eventually pass; for our logged run, all fifteen stages pass on the final attempt (100%).

Retries: Sum over stages of $\max(0, \text{attempts} - 1)$ for that stage.

Tool called: Whether the Researcher issued a `[SEARCH:]` tag during the run (binary, per task).

Tool success: Whether the Researcher’s tool observation was a successful extract rather than an error prefix, as recorded in JSON.

Task time: Wall-clock seconds for the full pipeline on a single machine for that run.

Mean judge score: Arithmetic mean of all fifteen per-stage scores.

TABLE III
PER-TASK EVALUATION RESULTS.

Task	P sc.	R sc.	W sc.	Tot. retr.	Tool called	Tool succ.	Time (s)
T1: Climate	9	9	9	0	Yes	Yes	151.6
T2: Nuclear	9	9	8	0	No	N/A	94.5
T3: COVID-19	9	9	9	0	Yes	No	110.9
T4: AI hist.	9	8	9	1	Yes	No	164.6
T5: WWI	9	9	9	0	No	N/A	102.0
Mean	9.0	8.8	8.8	0.2	3/5	1/3	124.7

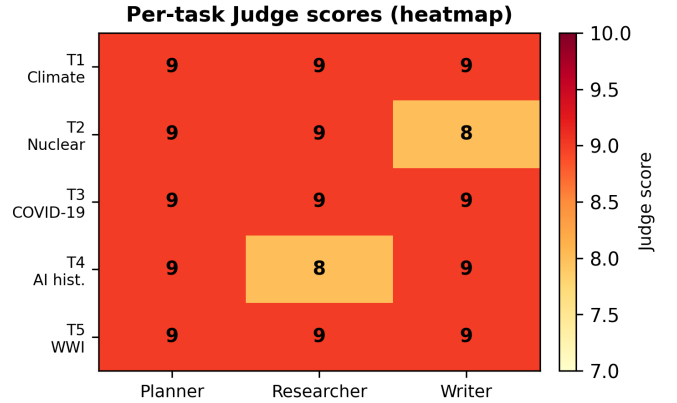


Fig. 3. Judge score heatmap across tasks and stages.

VII. RESULTS AND DISCUSSION

A. Per-Task Results

All numeric entries in Table III are taken directly from `eval_results.json` as produced by `evaluate.py`. Per-role means are ordinary arithmetic means over the five tasks. The tool-call fraction $3/5$ is the number of tasks in which the Researcher issued at least one `[SEARCH:]` request. The tool-success value $1/3$ is the number of those invocations in which the `tool_success` flag in the JSON log was `true` (a usable Wikipedia extract returned), divided by the number of tasks where the tool was called (three in this run), not by five.

Figure 3 visualizes the per-stage Judge scores from Table III as a heatmap (rows: T1–T5; columns: Planner, Researcher, Writer). Planner scores are uniformly high (all 9s), while variation is concentrated in the Researcher and Writer stages (two 8s). The two minima correspond to the Researcher on T4 and the Writer on T2: T4 reflects a retry triggered by an infrastructure issue (HTTP 429 leading to invalid Judge JSON), while T2’s lower Writer score is consistent with style redundancy rather than tool failure.

B. Score Analysis

The overall mean judge score is $133/15 \approx 8.87$. Per-role means are 9.0 for the Planner, 8.8 for the Researcher, and 8.8 for the Writer. Each of the Researcher and Writer received one score of 8 across the five tasks: the Researcher on T4 and the Writer on T2. Both cases reflect minor quality issues—

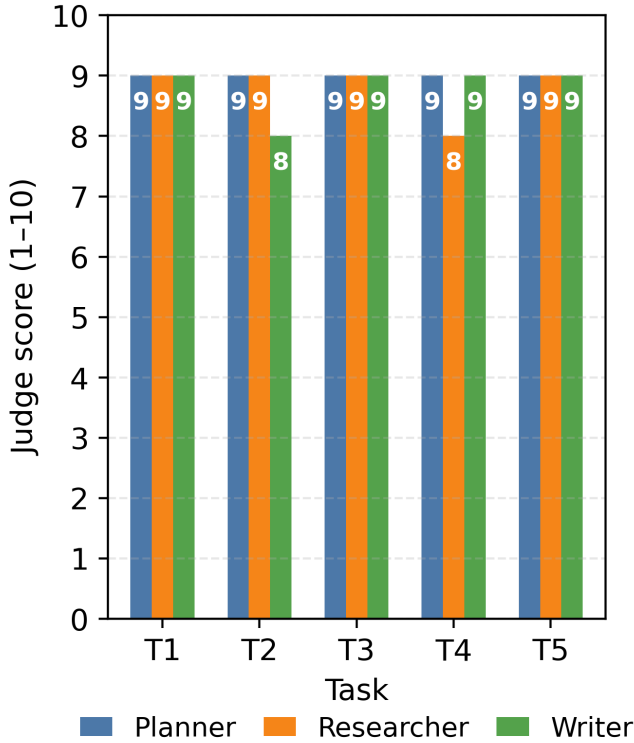


Fig. 4. Grouped bar chart of per-stage Judge scores for all five tasks. Planner scores are uniformly 9; the only below-maximum values are the Researcher on T4 (infrastructure-triggered retry) and the Writer on T2 (style redundancy). All scores lie in the range [8, 9].

incomplete coverage and style redundancy, respectively—rather than fundamental failures.

Figure 4 presents the same scores as a grouped bar chart, making the per-task, per-role distribution easy to compare side-by-side. The Planner bars are uniformly at 9 across all five tasks, confirming that decomposing the task into an actionable plan is a relatively straightforward step for the underlying model at this difficulty level. The two bars that fall to 8—the Researcher on T4 and the Writer on T2—are visually isolated as the only below-maximum values in the entire run. This concentration of variance in the Researcher and Writer stages is consistent with the expectation that synthesis and writing tasks are more sensitive to context quality than high-level planning. Overall the narrow [8, 9] range across all fifteen evaluations indicates that the pipeline produces consistently high-quality outputs for the five tested topics, while the two individual 8s identify the specific task-role combinations most likely to benefit from future improvement.

C. Retry Analysis

Only one retry was recorded across the entire evaluation: the Researcher stage of Task T4. During the first Researcher attempt, an HTTP 429 rate-limit response from Groq caused the Judge to return malformed JSON. The orchestrator’s fallback treated this as a failure and assigned a default score of 5 with `passed = false`, triggering a retry even though the

underlying Researcher output may have been acceptable. On the second attempt, with the Judge feedback string injected into context, the Researcher produced an output that passed with a score of 9. This case illustrates the infrastructure-failure pathway of the retry mechanism; the other four tasks completed with zero retries, and the pipeline never aborted.

D. Tool Integration Analysis

The tool was invoked on three tasks (T1, T3, T4). One invocation (T1, climate change) returned a usable Wikipedia extract; two (T3, T4) returned HTTP 404 error strings. The pipeline still completed in all cases because error strings are injected as [System] observations and the Researcher continues with its parametric knowledge. For stronger grounding, a search-then-resolve step (query → candidate titles → summary) is needed.

E. Execution Time

Wall-clock times range from 94.5 s to 164.6 s; the mean is 124.7 s. The slowest task is T4 (AI history) at 164.6 s, which is also the only task with a Researcher retry in this run, so extra LLM and Judge calls dominate the time. The 2 s client-side sleep before each completion call adds overhead proportional to the total number of API calls.

VIII. ETHICAL CONSIDERATIONS AND LIMITATIONS

A. Hallucination and Factual Accuracy

The Judge is itself an LLM; high scores indicate coherence and role fit, not verified factuality. Outputs should be human-checked before use in any high-stakes context.

B. Tool Reliability, APIs, and Grounding

Title-based Wikipedia access failed on two of the three Researcher tool queries in the logged run, while one call returned a valid summary. Improving retrieval (e.g., search-then-resolve a title) is an engineering and ethical priority when factual claims matter. Separately, Groq or Judge endpoints can return rate-limit errors (HTTP 429) or other transient failures, which can make the Judge return invalid JSON and force a stage retry—or, in the worst case, a low default parse score. This couples evaluation quality to provider behavior, a limitation of automated scoring beyond HTTP 404 errors from Wikipedia.

C. Bias and Autonomy

Models may encode cultural or geographic bias. Fully autonomous deployment without human oversight is inappropriate for medical, legal, or safety-critical domains.

IX. CONCLUSION AND FUTURE WORK

We presented a Judge-gated Planner–Researcher–Writer pipeline with Wikipedia tool hooks and a feedback-augmented retry mechanism. Experiments across five knowledge-synthesis tasks show a mean judge score of **8.87/10**, 100% pipeline completion, one retry at the Researcher stage of T4, and mixed tool success (1 of 3 calls returned a usable Wikipedia extract). Deterministic role assignment provides predictable task

decomposition, and the Judge gating layer enables autonomous quality control without human intervention.

Three directions stand out for future work. First, replacing the title-based Wikipedia lookup with a two-step search-then-resolve approach would enable reliable grounded retrieval. Second, introducing adaptive role management—where the Coordinator modifies a persistently underperforming agent’s system prompt at runtime—would address the rigidity of static assignment. Third, adding human evaluation as a secondary quality signal and running the evaluation over multiple seeds would provide a more credible and reproducible performance baseline.

ACKNOWLEDGMENT

The authors thank the Department of Data Science and Artificial Intelligence at the National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad, for the academic framework supporting this project.

REFERENCES

- [1] Z. Wang, S. Cai, A. Liu, X. Ma, and Y. Liang, “A survey on large language model-based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] T. Deng, F. Feng, J. Lv, Y. Li, and X. He, “A survey on multi-agent reinforcement learning for large language models,” *IEEE Trans. Neural Netw. Learn. Syst.*, 2024.
- [3] W. Chen et al., “AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents,” in *Proc. ICLR*, Vienna, 2024.
- [4] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proc. NeurIPS*, Vancouver, 2024.
- [5] C. Qian et al., “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. ICLR*, Vienna, 2024.
- [6] G. Li et al., “CAMEL: Communicative agents for ‘mind’ exploration of large language model society,” in *Proc. NeurIPS*, New Orleans, 2023.
- [7] S. Gronauer and K. Diepold, “Multi-agent deep reinforcement learning: A survey,” *Artificial Intelligence Review*, vol. 55, no. 2, pp. 895–943, 2022.
- [8] X. Liu et al., “AgentBench: Evaluating LLMs as agents,” in *Proc. ICLR*, Vienna, 2024.